

Réponses au Livrable 1 — Plan de sécurité et budget (ECUE2)

Ce document compile les réponses aux exigences de conception et de sécurité définies dans le cadre du Livrable 1 du projet **Wave Credit Score**. (À exporter en PDF pour le rendu final).

1. Présentation du projet

- **Problème résolu et utilisateurs ciblés** : De nombreux citoyens en Afrique de l'Ouest ne possèdent pas de base de données bancaire exploitable par les institutions financières classiques, mais ils réalisent un grand nombre de transactions journalières via Mobile Money (Wave, Orange Money, etc.). Le projet *Wave Credit Score* vise à fournir un outil d'importation de relevés "Wave" au format PDF, d'en faire l'analyse et d'y appliquer un modèle d'évaluation de la solvabilité. Cette application est ciblée sur les banques (évaluation du risque) ou sur le particulier (connaître sa capacité d'emprunt réelle).
- **Modèle ML envisagé** : L'extraction des informations est couplée à un modèle d'évaluation par RandomForest/Régression ou algorithmes heuristiques basés sur : Ratio d'Épargne, Fréquence, Revenu moyen et Régularité des revenus. Le modèle exporté (Pickle/Joblib) sera relativement léger (<100MB) optimisant ainsi les coûts mémoire du conteneur.
- **Plateforme cloud pressentie : AWS (Amazon Web Services)**. Motivé par l'excellente offre "Free Tier" et le respect des standards de sécurité : Amazon ECR pour le registre d'images, Amazon ECS (via AWS Fargate) ou App Runner pour exécuter l'API sans gérer les serveurs (Serverless), et GitHub Actions pour l'orchestration continue (CI/CD) liée via OIDC (OpenID Connect).

2. Schéma d'architecture prévue

(La schématisation complète avec un outil de type Draw.io/Excalidraw est préconisée. Voici l'explication sous format texte) :

- **Zone publique Internet** : L'utilisateur interagit avec le **Frontend (Angular 16)** servi via Nginx, accessible en HTTPS sur un domaine public. L'authentification OAuth (Google Auth) se passe côté navigateur communiquant avec les serveurs d'identité de Google.
- **Zone Privée/Backend (VPC AWS)** : Le Frontend communique via HTTPS avec l'API Web (**FastAPI - Backend**). L'API est la seule à contenir l'orchestration du scoring machine learning et le décodage du jeton JWT.
- **Storage** : L'API discute avec la **Base de données relationnelle AWS RDS (PostgreSQL)** en réseau interne privé. (Aucun accès direct de l'extérieur vers la base de données).

3. Analyse des risques

Risque	Impact	Probabilité	Mesure prévue
Fuite de secrets/API Keys dans le repository Git	Élevé	Moyenne	Maintien vigilant du .gitignore pour le .env. Scan continu CI avec Trufflehog

Vol de session via Faux-Token	Élevé	Moyenne	Validation des ID Tokens via la librairie officielle google-auth Python.
Injection SQL ou Corruption Data	Élevé	Faible	Utilisation stricte d'un ORM (SQLAlchemy) et validation fine des Payload via Pydantic.
Image Docker Vulnérable (CVD)	Moyen	Élevée	Analyse automatique poussée par le pipeline (Trivy) sur les criticité HIGH/CRITICAL avec blocage automatique au déploiement en cas d'alerte.
Explosion du budget Cloud	Moyen	Moyenne	Création d'alertes via AWS Budgets. Conteneurs bloqués sur une contrainte max RAM/vCPU (pas d'auto-scaling excessif)
Attaque via PDD malicieux (Zip-Bomb/DoS)	Élevé	Faible	Définition de Timeout pour le parseur Backend de pdfminer / pypdf, limitation de taille de fichier pour les uploads NGINX/FastAPI.

4. Gestion des accès (IAM)

- **Rôles de déploiement (CI/CD GitHub Actions)** : Le workflow possède les autorisations temporelles et non pérennes (Least Privilege) pour se connecter à AWS en assumant le rôle `AWS_ROLE_TO_ASSUME` par authentification par certificat OIDC (OIDC évite d'exporter des clés longues durées vers GitHub).
- **Politique de service ECS/AppRunner** : Le conteneur backend reçoit exclusivement des droits accès réseau interne lui permettant de joindre PostgreSQL, sans droits de modifier d'autres ressources AWS.
- **Utilisateurs Finaux** : Aucune gestion par mots de passe locale (délégué à Google OAuth), réduisant considérablement la surface d'attaque lié aux identifiants.

5. Estimation des coûts

Hypothèse de base (Phase de MVP) : Environ ~100 à ~300 utilisateurs finaux effectuant 2 à 3 prévisions de scoring par mois. Faible trafic réseaux :

Poste	Service	Coût mensuel estimé
Compute (Backend ML)	AWS ECS Fargate (0.25 vCPU, 0.5 GB) ou AppRunner	~ 4 \$
Hébergement Frontend	Nginx dans ECS (Idem Fargate) ou S3/CloudFront	~ 1 \$
Base de données	AWS RDS (db.t4g.micro) ou Amazon RDS Free Tier	~ 0 \$ à 15 \$*
Stockage image conteneurs	Amazon ECR	~ 0,5 \$
Bande Passante (Egress)	AWS Data Transfer (Bas volume : < 1GB)	~ 0 \$ (Inclus)

Total Estimé		~ 5.5 \$ (Avec Free Tier applicable)
--------------	--	--------------------------------------

Note sur RDS : Les mois de développement avec compte AWS Free Tier, l'instance Single-AZ t4g/t3.micro sera gratuite les 12 premiers mois.

6. Plan de mise en œuvre (Checklist de Sécurité CI/CD & Déploiement)

- ☒ Initialiser un fichier `.gitignore` robuste et séparer les credentials dans un `.env`.
- ☒ Intégrer les Scans Vulnérabilités (CI/CD) sur toute nouvelle Push : `trufflehog` (Credentials/Secrets), `pip-audit` (Dépendances toxiques Python), et `Trivy` (Images Docker).
- ☒ Activer HTTPS sur l'API et le portail (Enjeu de Déploiement Cloud sur AWS via Amazon ACM Certificats).
- ☒ Implémenter l'authentification sécurisée des méthodes API via la vérification par clé via "Depend" OAuth `X-API-Key` ou `Bearer Token JWT`.
- ☒ Mettre un garde-fou au budget Cloud via des notifications Alert Billing (AWS Budgets) à 5\$ / mois.